EDAGenerate 项目介绍与实现说明

1. 项目背景

本项目的目标，是做一个“自然语言描述 -> 电路结构化结果 -> 原理图可视化”的演示型系统。

更具体地说，输入是一段中文提示词，例如“设计一个 RC 低通滤波电路”，输出不是自由文本，而是一个结构化字典，里面明确给出：

- 电路功能 `function`
- 节点信息 `components`
- 节点之间的连接关系 `edges`
- 节点坐标 `x / y`

在这个基础上，再通过绘图脚本把结构化字典渲染成一张可读的原理图图片。

这个项目的重点不在于做一个覆盖真实工业场景的 EDA 系统，而在于验证这样一条技术路线是否可行：

1. 用受控方式构建高质量训练数据。
2. 用轻量的文本生成模型学习“提示词到结构化电路”的映射。
3. 把模型输出继续转换成图像结果，形成完整演示闭环。

2. 项目核心思路

2.1 先做数据，不先做大模型

项目一开始就明确了一个关键判断：

这个任务的上限主要由数据集质量决定，而不是由模型规模决定。

原因很直接：

- 输出是结构化电路字典，不是普通自然语言。
- 电路连接关系必须正确，不能只做到“看起来像”。
- 节点坐标要有一致的布局风格，否则模型会把坐标学成噪声。

因此，这个项目优先解决的是“如何设计数据集”，而不是直接去堆更大的模型。

2.2 数据集采用 `prompt -> 标准化字典`

最终采用的数据形式是：

- 输入：自然语言提示词 `prompt`
- 输出：标准化结构字典 `output`

标准化字典的核心结构为：

```
{
"function": "RC低通滤波",
"layout_type": "horizontal_main_vertical_branch",
"components": {
"VIN": {"type": "input", "x": 10, "y": 20},
"R1": {"type": "resistor", "x": 40, "y": 20},
"OUT": {"type": "output", "x": 70, "y": 20},
"C1": {"type": "capacitor", "x": 70, "y": 50},
"GND": {"type": "ground", "x": 70, "y": 80}
},
"edges": {
"edge1": ["VIN", "R1"],
"edge2": ["R1", "OUT"],
"edge3": ["OUT", "C1"],
"edge4": ["C1", "GND"]
}
}
```

之所以不用“每条边里重复写节点坐标”的格式，而采用“组件表 + 边表”的格式，是因为这种结构更稳定：

- 同一节点坐标只存一次。
- 更方便程序校验。
- 更方便模型训练后的解析和渲染。
- 后续也更容易转成图结构或其他格式。

2.3 先做模板，再做大规模数据

本项目没有采用“手工写几万条数据”的路线，而是采用“金模板 + 程序扩增”的路线。

具体分为两层：

1. 金模板层

负责定义功能、电路节点、连接关系和布局类型。

2. 样本扩增层

通过程序生成 prompt 变体、坐标变体和布局变体，形成大规模训练样本。

这条路线的目的，是先保证知识骨架正确，再扩大样本规模，而不是先追求数量。

3. 已完成的数据工程工作

3.1 模板库构建

项目中先构建了一批不带坐标的结构模板，用于定义基础电路拓扑。

随后又进一步筛选出第一版金模板，并扩展成 V2 模板库：

- `eda_gold_templates_v1_120.json`
- `eda_template_bank_v2_480.json`

V2 模板库的目标，是作为后续生成 4 万到 5 万条训练数据的结构母体。

当前覆盖的基础功能主要包括：

- LED 指示
- RC 低通滤波
- RC 高通滤波
- LC 滤波
- 电源去耦
- 电阻分压
- 上拉按键输入
- 下拉按键输入
- 二极管防反接
- 简单整流滤波

3.2 程序化数据生成

已经实现了基于模板库的训练数据生成脚本：

- `scripts/generate_training_dataset.py`

该脚本会完成以下工作：

1. 读取模板库。

2. 根据模板类型生成受控 prompt。
3. 按布局规则生成节点坐标。
4. 组织成统一 JSONL 样本格式。
5. 输出训练集文件。

当前已经生成出一版大规模训练集：

- `datasets/eda_train_50000.jsonl`

这批数据不是完全自由生成，而是通过模板、命名、布局和坐标规则受控扩增得到的，因此更适合做结构化学习任务。

3.3 数据质量控制

数据生成过程中考虑了基本的合法性和一致性校验，例如：

- 必需字段是否存在
- 边是否引用了不存在的节点
- 同一节点坐标是否统一
- 边编号是否连续
- 坐标是否存在明显冲突

这一步的目的，是尽量避免“表面上像 JSON，实际上结构错误”的脏数据进入训练集。

4. 模型设计与训练实现

4.1 模型选择

项目没有从零实现 Transformer，而是采用 Hugging Face `transformers` 生态中的预训练模型进行微调。

最终选用的是：

- `google/mt5-small`

选择这个模型的原因有三点：

1. 支持中文输入输出。
2. 适合做“文本到结构化文本”的 seq2seq 任务。

3. 参数规模相对可控，更适合演示型项目和有限算力环境。

4.2 训练形式

训练任务定义为：

- 输入：中文 prompt
- 输出：结构化 JSON 字符串

项目中已经完成：

- 数据加载
- tokenizer 长度统计
- mT5 微调 notebook
- Hugging Face Hub 自动推送

相关文件包括：

- `mt5_small_train_kaggle_hub.ipynb`
- `scripts/measure_token_lengths.py`

训练时还做了这些实际工程处理：

- 根据真实 token 长度分布设置 `max_input_length` 和 `max_target_length`
- 考虑了早停策略
- 支持将训练结果推送到 Hugging Face 仓库

当前训练后的模型仓库为：

- `wzmmmm/mt5-eda-generate`

4.3 本地推理验证

为了验证模型确实能在本地环境使用，项目中还实现了 CPU 推理脚本：

- `scripts/download_hf_model.py`
- `scripts/run_inference_cpu.py`

这部分已经完成的能力包括：

• 从 Hugging Face 拉取模型到本地

• 加载 tokenizer 和权重

• 输入中文 prompt 做推理

• 提取 JSON

• 对模型输出进行基础结构校验

实际测试中，模型已经能够输出类似 RC 低通这类基础电路的合法结构字典。

5. 原理图渲染与演示实现

5.1 从结构字典生成图片

本项目不仅做到了“输出结构化结果”，还实现了“把结构化结果渲染成原理图图片”。

核心脚本包括：

• `scripts/render_schematic.py`

• `scripts/render_from_prediction.py`

实现逻辑是：

1. 接收标准化字典。
2. 按节点坐标放置节点。
3. 根据 `edges` 画连接线。
4. 在图片中标注节点名称。
5. 输出为 PNG 图片。

5.2 中文标签优化

考虑到项目使用者不一定有电子电路背景，渲染模块还进一步做了节点中文化处理。

例如：

• `VIN` 显示为“输入 (VIN)”

• `R1` 显示为“电阻 (R1)”

• `C1` 显示为“电容 (C1)”

• `GND` 显示为“接地 (GND)”

这一步显著提升了演示时的可读性，使得即使不熟悉电子元件英文缩写，也能快速理解图中每个节点的含义。

5.3 Notebook 演示闭环

最终，项目选择用 Notebook 来完成演示，而不是继续推进完整前后端部署。

当前已经形成了一个完整的演示 Notebook：

- `mt5_notebook_interactive_demo.ipynb`

这个 Notebook 可以完成：

1. 加载本地训练好的模型。
2. 手动输入提示词，或从 20 条备选提示词中选择。
3. 执行 LLM 推理。
4. 解析模型输出 JSON。
5. 调用外部脚本生成图片。
6. 直接在 Notebook 中展示结果图。

这条链路已经构成了一个完整、可展示的项目 Demo。

6. 项目落地过程中的关键决策

6.1 不直接让模型学习“任意原理图”

项目一开始就限定了问题边界，只做：

- 基础功能电路
- 小规模结构
- 有限元件集合
- 规则布局

这保证了任务可以在有限时间内做成闭环，而不会因为问题边界过大而失控。

6.2 坐标优先程序生成，而不是完全交给模型自由学习

项目中对坐标的判断很明确：

坐标更像布局规则，而不是电路知识本体。

因此，前期主要通过规则方式生成坐标，而不是寄希望于模型在小规模数据上自动学会高质量布局。

这使得训练集中的空间分布更一致，也让渲染结果更稳定。

6.3 先做可解释、可校验的结构输出

与“直接输出图片”相比，结构化字典的优势在于：

- 更容易检查对错
- 更容易调试
- 更容易统计
- 更容易被后续程序利用

所以项目选择先把“文本 -> 结构字典”做通，再把字典渲染成图片，而不是一上来就做图像生成。

7. 当前项目已经实现了什么

截至目前，项目已经完成了这些核心成果：

1. 完成了任务边界定义和数据 schema 设计。
2. 构建了基础功能电路模板库与金模板库。
3. 实现了程序化样本生成，产出 5 万条训练数据。
4. 完成了 mT5-small 的训练方案和训练 notebook。
5. 将训练模型推送到了 Hugging Face。
6. 实现了本地 CPU 推理脚本。
7. 实现了结构字典到图片的渲染能力。
8. 完成了基于 Notebook 的交互式演示链路。

换句话说，本项目已经从“想法”走到了“可运行 Demo”的阶段。

8. 项目价值与后续方向

这个项目的价值主要体现在两点：

第一，它验证了“自然语言到结构化电路表示”的路线是可落地的。

第二，它把数据、训练、推理、渲染和展示都串成了一条完整闭环。

如果后续继续推进，可以重点考虑以下方向：

- 扩大模板族和功能覆盖面
- 增加更复杂的拓扑变体
- 强化输出 JSON 的鲁棒性
- 改进渲染效果，使其更接近真实原理图风格
- 将 Notebook Demo 升级为完整服务化推理系统

9. 总结

EDAGenerate 不是一个单点脚本，而是一条完整的小型工程链路。

它从数据集设计出发，通过模板化和程序化方式生成训练样本，使用 `mT5-small` 完成结构化电路生成，再将模型输出渲染成图片，最终用 Notebook 形成可交互的演示结果。

这个项目已经证明了这样一个方向：

“在受控任务边界内，轻量模型 + 高质量合成数据 + 规则化渲染”是可以做出可展示、可解释、可扩展的 EDA 生成 Demo 的。